

Spring Boot Configurati

`application.properties`

`@Value` ,

`@ConfigurationProperti`

Runner Interfaces

1. Why Do We Need Configuration?

In real applications, not every value should be hardcoded inside Java classes.

For example:

```
@Component
public class PaymentService {

    private String providerName = "Razorpay";
    private int retryCount = 3;

    public void pay() {
        System.out.println("Payment done using " + providerName);
        System.out.println("Retry count: " + retryCount);
    }
}
```

This code works, but the values are fixed inside the Java source code.

Now imagine that tomorrow we want to change the payment provider from:

Razorpay

to:

```
Stripe
```

If the value is hardcoded, we would need to:

1. Change the Java code
2. Recompile the project
3. Rebuild the application
4. Redeploy the application

For values that may change across environments or over time, this is not a clean approach.

Examples of such values:

```
payment provider name  
retry count  
timeout value  
feature enabled/disabled flag  
database URL  
API key  
server port  
external service URL
```

These values are better kept outside the main business logic.

This is where configuration files come in.

2. What Is `application.properties` ?

`application.properties` is a key-value configuration file that Spring Boot automatically reads by convention.

The default location is:

```
src/main/resources/application.properties
```

Example:

```
payment.provider=Razorpay  
payment.retry-count=3  
payment.enabled=true  
payment.timeout=5000
```

Here, the values are no longer written directly inside the Java class.

They are now configuration values.

Spring Boot automatically loads this file when the application starts.

Spring Boot also supports YAML configuration using:

```
application.yml
```

Example:

```
payment:  
  provider: Razorpay  
  retry-count: 3  
  enabled: true  
  timeout: 5000
```

Both `application.properties` and `application.yml` are used for configuration. The difference is mainly in the writing style.

3. Important Point: Is `application.properties` Really External?

When `application.properties` is placed inside:

```
src/main/resources
```

it becomes part of the application build.

That means it is packaged inside the final JAR file.

So technically, if we only change this internal file in the source to rebuild the application.

But Spring Boot provides externalized configuration, which means it can also be supplied or overridden from outside the package.

For example, Spring Boot can read configuration from:

```
application.properties  
application.yml  
environment variables  
command-line arguments  
system properties  
external config files
```

So the main idea is:

Keep changeable values outside Java business logic so the code remains clean and flexible.

4. What Is Externalized Configuration?

Externalized configuration means keeping configuration values outside the Java code.

Instead of writing this:

```
private String providerName = "Razorpay";
```

we write the value in a configuration file:

```
payment.provider=Razorpay
```

Then we inject that value into the Java class.

This makes the application easier to manage across different environments.

For example:

Development Environment

```
payment.provider=TestProvider  
payment.retry-count=1
```

Production Environment

```
payment.provider=Razorpay  
payment.retry-count=3
```

The Java code can remain the same.

Only the configuration changes.

5. Using Configuration Values with `@Value`

Once values are present in `application.properties`, we need a way to use them inside Java classes.

One simple way is to use:

```
@Value
```

Example:

```
@Component  
public class PaymentService {  
  
    private final String providerName;  
    private final int retryCount;  
  
    public PaymentService(  
        @Value("${payment.provider}") String providerName,  
        @Value("${payment.retry-count}") int retryCount)  
    {  
    }  
}
```

```

        this.providerName = providerName;
        this.retryCount = retryCount;
    }

    public void pay() {
        System.out.println("Payment done using
me);
        System.out.println("Retry count: " + retryCount);
    }
}

```

Here:

```
@Value("${payment.provider}")
```

means:

Go to the Spring Environment, find the property named `payment.provider`, and inject its value here.

Similarly:

```
@Value("${payment.retry-count}")
```

means:

Find the property named `payment.retry-count` and inject it into this parameter.

6. Default Value with `@Value`

Sometimes a property may be missing from `application.properties`.

If the property is missing and no default value is provided, the application may fail during startup.

To avoid this, we can provide a default value.

Example:

```
@Value("${payment.provider:DefaultProvider}")  
private String providerName;
```

Here:

```
${payment.provider:DefaultProvider}
```

means:

Use `payment.provider` if it is available.
If it is not available, use `DefaultProvider`.

Constructor example:

```
@Component  
public class PaymentService {  
  
    private final String providerName;  
  
    public PaymentService(  
        @Value("${payment.provider:DefaultProvider}") String  
        providerName) {  
        this.providerName = providerName;  
    }  
}
```

This is useful when a property is optional.

7. Problem with Too Many @Value Annotations

`@Value` is simple and useful for small cases.

But imagine we have many related properties:

```
payment.provider=Razorpay
payment.retry-count=3
payment.enabled=true
payment.timeout=5000
```

If we use `@Value`, we need to inject every value separately:

```
@Value("${payment.provider}")
private String provider;

@Value("${payment.retry-count}")
private int retryCount;

@Value("${payment.enabled}")
private boolean enabled;

@Value("${payment.timeout}")
private int timeout;
```

This works, but it becomes messy when the number of properties increases.

For grouped configuration, Spring Boot provides a cleaner option:

```
@ConfigurationProperties
```

8. What Is `@ConfigurationProperties` ?

`@ConfigurationProperties` is used to bind a group of related configuration properties to a Java object.

Instead of injecting every property separately, we create one configuration class.

Example `application.properties` :

```
payment.provider=Razorpay
payment.retry-count=3
```

```
payment.enabled=true  
payment.timeout=5000
```

Now create a Java class:

```
package in.strikes.config;  
  
import org.springframework.boot.context.properties.ConfigurationProperties;  
import org.springframework.stereotype.Component;  
  
@Component  
@ConfigurationProperties(prefix = "payment")  
public class PaymentProperties {  
  
    private String provider;  
    private int retryCount;  
    private boolean enabled;  
    private int timeout;  
  
    public String getProvider() {  
        return provider;  
    }  
  
    public void setProvider(String provider) {  
        this.provider = provider;  
    }  
  
    public int getRetryCount() {  
        return retryCount;  
    }  
  
    public void setRetryCount(int retryCount) {  
        this.retryCount = retryCount;  
    }  
}
```

```

public boolean isEnabled() {
    return enabled;
}

public void setEnabled(boolean enabled) {
    this.enabled = enabled;
}

public int getTimeout() {
    return timeout;
}

public void setTimeout(int timeout) {
    this.timeout = timeout;
}
}

```

The prefix is:

```
@ConfigurationProperties(prefix = "payment")
```

This means Spring Boot will look for all properties starting with:

```
payment
```

Then it will bind them to this class.

Mapping:

```

payment.provider      → provider
payment.retry-count   → retryCount
payment.enabled       → enabled
payment.timeout       → timeout

```

Now this configuration class can be injected wherever needed.

Example:

```

@Component
public class PaymentService {

    private final PaymentProperties paymentPro

    public PaymentService(PaymentProperties pa
s) {
        this.paymentProperties = paymentProperties;
    }

    public void pay() {
        System.out.println("Payment done using " + paymentPro
perties.getProvider());
        System.out.println("Retry count: " + paymentPropertie
s.getRetryCount());
        System.out.println("Enabled: " + paymentProperties.is
Enabled());
        System.out.println("Timeout: " + paymentProperties.ge
tTimeout());
    }
}

```

This is cleaner because all payment-related configuration is stored in one object.

9. Why Does `payment.retry-count` Map to `retryCount` ?

This happens because of relaxed binding.

Spring Boot is flexible with property names.

So this property:

```
payment.retry-count=3
```

can bind to this Java field:

```
private int retryCount;
```

Spring Boot understands different naming formats such as:

```
retry-count  
retryCount  
retry_count  
RETRY_COUNT
```

For beginners, the simplest way to explain it is:

Spring Boot can automatically convert common property naming styles into Java field naming style.

In Java, we usually write field names in camelCase:

```
retryCount
```

In properties files, we usually write names in kebab-case:

```
retry-count
```

Spring Boot can connect both of them.

10. @Value vs @ConfigurationProperties

@Value

Use `@Value` when you need one or two simple values.

Example:

```
@Value("${payment.provider}")  
private String provider;
```

Good for:

```
single value
small demo
simple property injection
```

But it becomes repetitive when many related properties are n

@ConfigurationProperties

Use `@ConfigurationProperties` when many related values belong to the same group.

Example:

```
payment.provider=Razorpay
payment.retry-count=3
payment.enabled=true
payment.timeout=5000
```

Good for:

```
grouped configuration
cleaner code
large applications
better readability
```

Simple rule:

For one or two values, `@Value` is fine.

For a group of related values, prefer `@ConfigurationProperties`.

11. Why Do We Need Runner Interfaces?

In a web application, code usually runs when an HTTP request comes.

For example:

```
Browser/Postman sends request
↓
```

```
Controller receives request
↓
Service method executes
```

But in this lecture, we are not using:

```
spring-boot-starter-web
controller
endpoint
browser request
Postman request
```

So we need a way to run some code after the Spring Boot application starts.

Spring Boot gives us two common interfaces for this:

```
CommandLineRunner
ApplicationRunner
```

These interfaces allow us to run code after the Spring container is ready.

12. Why Not Call the Bean Manually from `main()` ?

Technically, we can do this:

```
public static void main(String[] args) {
    ConfigurableApplicationContext context =
        SpringApplication.run(SpringBootCoreDemoApplicati
on.class, args);

    PaymentService paymentService = context.getBean(PaymentSe
rvice.class);
    paymentService.pay();
}
```

This works.

But this should not be the main approach in a Spring Boot ap

Why?

Because it brings us back to a manual style:

```
start context manually  
fetch bean manually  
call method manually
```

A better Spring Boot style is:

| Let Spring create a bean that runs automatically after startup.

That is where runner interfaces help.

13. **CommandLineRunner**

CommandLineRunner is a Spring Boot interface used to run code after the Spring application has started.

Example:

```
package in.strikes.runner;  
  
import in.strikes.service.PaymentService;  
import org.springframework.boot.CommandLineRunner;  
import org.springframework.stereotype.Component;  
  
@Component  
public class AppRunner implements CommandLineRunner {  
  
    private final PaymentService paymentService;  
  
    public AppRunner(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

```

    }

    @Override
    public void run(String... args) {
        paymentService.pay();
    }
}

```

What happens here?

```

Spring creates AppRunner as a bean
↓
Spring injects PaymentService into AppRunner
↓
Spring Boot sees that AppRunner implements CommandLineRunner
↓
Spring Boot calls the run() method after startup
↓
paymentService.pay() executes

```

The method signature is:

```
public void run(String... args)
```

Here, `String... args` means the method can receive multiple command-line arguments as strings.

14. Passing Arguments to `CommandLineRunner`

Suppose we run the application from terminal like this:

```
java -jar app.jar hello world
```

Then inside `CommandLineRunner`:

```

@Override
public void run(String... args) {
    for (String arg : args) {
        System.out.println(arg);
    }
}

```

Output:

```

hello
world

```

`CommandLineRunner` receives arguments as a simple string array.

15. `ApplicationRunner`

`ApplicationRunner` is similar to `CommandLineRunner`.

The difference is that it provides arguments in a more structured form using:

`ApplicationArguments`

Example:

```

package in.strikes.runner;

import in.strikes.service.PaymentService;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class AppStartupRunner implements ApplicationRunner {

    private final PaymentService paymentService;

```

```

    public AppStartupRunner(PaymentService pay
        this.paymentService = paymentService;
    }

    @Override
    public void run(ApplicationArguments args)
        paymentService.pay();
    }
}

```

Here, the method signature is:

```

public void run(ApplicationArguments args)

```

Instead of raw string arguments, Spring Boot gives us an `ApplicationArguments` object.

This object can understand option arguments more clearly.

For example, if we run:

```

java -jar app.jar --provider=Razorpay --retry=3

```

`ApplicationRunner` can read these as named options.

Example:

```

@Override
public void run(ApplicationArguments args) {
    System.out.println(args.getOptionValues("provider"));
    System.out.println(args.getOptionValues("retry"));
}

```

So the simple difference is:

```

CommandLineRunner → gives raw String arguments

```

ApplicationRunner → gives structured ApplicationArguments

16. Complete Startup Flow

A simplified Spring Boot startup flow looks like this:

```
main() method starts
↓
SpringApplication.run() executes
↓
Spring Boot prepares the Environment
↓
Configuration values are loaded
(application.properties, application.yml, environment variables,
command-line arguments, etc.)
↓
ApplicationContext is created
↓
@SpringBootApplication is processed
↓
@ComponentScan scans the package
↓
@Component / @Service / @Repository / @Controller classes are
discovered
↓
@EnableAutoConfiguration checks dependencies on the classpath
↓
Matching auto-configurations are applied
↓
Beans are created
↓
Dependencies are injected
↓
Configuration values are injected or bound
```

```
(@Value / @ConfigurationProperties)
↓
CommandLineRunner / ApplicationRunner executes
↓
Application either exits or keeps running depending on the configuration of application
```

In simple words:

Spring Boot starts the container, loads configuration, scans classes, creates beans, injects dependencies, applies auto-configuration, and finally runs startup logic.

17. Final Summary

application.properties

application.properties is used to keep configuration values in key-value format.

Example:

```
payment.provider=Razorpay
payment.retry-count=3
```

Externalized Configuration

Externalized configuration means keeping changeable values outside Java business logic.

This helps us avoid hardcoding values inside classes.

@Value

@Value is used to inject a single configuration value.

Example:

```
@Value("${payment.provider}")  
private String provider;
```

It is simple and useful for small cases.

@ConfigurationProperties

`@ConfigurationProperties` is used to bind a group of related properties to a Java object.

Example:

```
@ConfigurationProperties(prefix = "payment")
```

It is cleaner when many related configuration values are present.

CommandLineRunner

`CommandLineRunner` runs code after the Spring Boot application has started.

It receives command-line arguments as raw strings.

```
run(String... args)
```

ApplicationRunner

`ApplicationRunner` also runs code after startup, but it receives arguments in a structured form.

```
run(ApplicationArguments args)
```

Spring Boot does not force us to hardcode everything inside Java classes.

We can keep changeable values in configuration files, inject them into beans, and run startup logic using runner interfaces when there is no web request involved.